

# Apuntes Inteligencia Artificial

Santiago Chaves Garbanzo  
2023047710

Escuela de Ingeniería en Computación  
Tecnológico de Costa Rica

Profesor: Steven Pacheco Portuquez  
Curso: IC6200 Inteligencia Artificial  
Fecha: 21 de abril de 2026

**Resumen**—En estos apuntes se repasan conceptos introductorios de clasificación con redes neuronales a partir del caso de MNIST. Se revisa el paso de regresión logística binaria a clasificación multiclase, el uso de one-hot encoding, la representación matricial de una capa y la motivación para usar capas intermedias no lineales en redes neuronales. Además, se resumen los lineamientos generales del proyecto del curso, incluyendo el uso del dataset Speech Commands, la técnica de aumento de datos SpecAugment, el diseño de modelos convolucionales, el preprocesamiento mediante espectrogramas, la evaluación de resultados y la exportación del modelo a ONNX para su integración en una aplicación móvil.

## I. REPASO CLASE ANTERIOR

En la clase anterior se vio el dataset de MNIST, en el cual hay muestras de diferentes números.

Como ejemplo se vio que, si se eligen los píxeles de una imagen, se podría hacer una regresión logística para clasificar un solo número. Sería una entrada de 784, y se asigna un peso para cada entrada; entonces se tendría un modelo que podrá clasificar si una imagen es dicho número. En este caso solo se está clasificando un dígito en específico, como se muestra en la Figura 1.

Se debe recordar que esta es una regresión logística normal: una parte lineal, que es la interacción entre features y pesos; una función de activación, que es la que da el comportamiento no lineal; y luego una anidación de funciones que une a ambas.

Luego se pasó a cómo sería un sistema de clasificación de múltiples clases. Para ello, primero hay que pasar a usar one-hot coding, que consiste en encender el bit de la clase que representa cada feature, como se observa en la Figura 2.

Para entender el problema, primero se pasó a un plano absurdo, en el cual se imagina que un solo píxel clasificaría todo el dataset. Entonces, lo que se ocuparía sería tener 10 regresiones logísticas y que cada una detecte un dígito en una clase 3. De acuerdo con eso, se encendería el bit de la clase correspondiente y se apagarían los del resto.

Luego se vio qué pasaría con dos píxeles. En este caso, cada uno de ellos debe mandar información a las 10 regre-

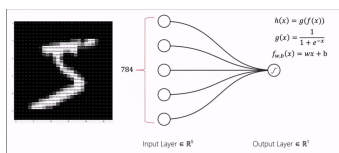


Figura 1. Ejemplo de regresión logística para MNIST

siones logísticas. Ahora, en cada una de ellas se tendrán dos pesos y un bias, cada uno orientado a detectar una clase en específico 4.

Cuando se empiece a aumentar la cantidad de píxeles, se verá algo así 5. Es decir, los 784 píxeles conectados a cada una de las regresiones logísticas (neuronas). Son una gran cantidad de pesos, se da el ejemplo sobre que es como pasar una fotocopia de la imagen a cada una de las neuronas para saber si saben o no reconocer dicha imagen.

Ahora bien, para realizar el cálculo de cada capa para evitar hacer operaciones iterativas, se empleará álgebra matricial. En lugar de tener solo una regresión  $Wx + B$ , se tendrá una matriz que represente toda la capa. Teniendo eso, solo se debe tomar el  $X$  de la entrada (los 784 píxeles), multiplicarlo por la matriz  $W$  (los pesos de todas las neuronas de la capa) y luego sumarle un vector de bias. Esto dará un vector con los resultados de cada una de las regresiones logísticas, como se representa en la Figura 6.

Por otro lado, se comentó que en este caso se tiene una entrada lineal que se conecta con 10 regresiones logísticas, y que eso en el fondo sigue siendo un problema lineal. Se está usando una recta para determinar el comportamiento. Por lo tanto, si se ocupa resolver un escenario no lineal, esas neuronas aún siguen sin tener esa capacidad.

Por lo tanto, si se vuelve a pensar en el problema binario de clasificar si un número es un cinco o no lo es, antes lo que se hacía era usar una sola regresión logística, a la cual llegan todas las entradas  $X$ , y a partir de ahí se toma una

| class | one-hot codification |
|-------|----------------------|
| 0     | 1 0 0 0 0 0 0 0 0 0  |
| 1     | 0 1 0 0 0 0 0 0 0 0  |
| 2     | 0 0 1 0 0 0 0 0 0 0  |
| 3     | 0 0 0 1 0 0 0 0 0 0  |
| 4     | 0 0 0 0 1 0 0 0 0 0  |
| 5     | 0 0 0 0 0 1 0 0 0 0  |
| 6     | 0 0 0 0 0 0 1 0 0 0  |
| 7     | 0 0 0 0 0 0 0 1 0 0  |
| 8     | 0 0 0 0 0 0 0 0 1 0  |
| 9     | 0 0 0 0 0 0 0 0 0 1  |

Figura 2. Ejemplo de one-hot encoding

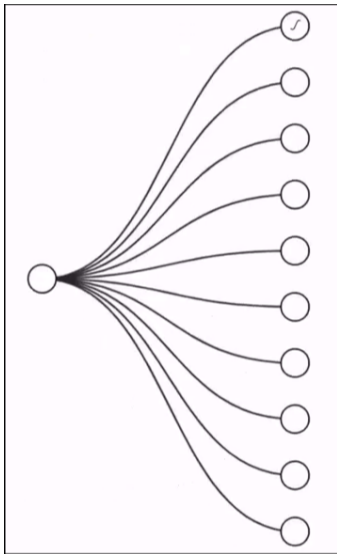


Figura 3. Ejemplo de una entrada

decisión. Pero ahora se hará distinto: se va a pasar la entrada a una capa con múltiples neuronas, y luego se pasarán esos resultados a otra capa con una sola regresión logística, que es la que tomará la decisión de si se tiene o no un 5. La ventaja es que la salida de la primera capa ya es no lineal, y al dársela a la siguiente regresión logística, permite algo muy poderoso, porque se está alimentando una nueva neurona con comportamiento no lineal. Entonces, esta clasificación binaria sería más compleja que una regresión logística clásica.

Eso sería en caso de usar una red neuronal para una clasificación binaria, en la cual hay una capa de entrada que define el tamaño inicial de la arquitectura; luego, la capa intermedia es todo el cómputo que se haga entre la entrada y la salida; y la salida es ya sea una salida binaria, como una

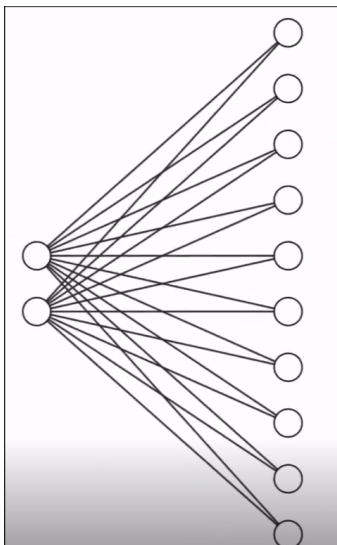


Figura 4. Ejemplo de dos entradas

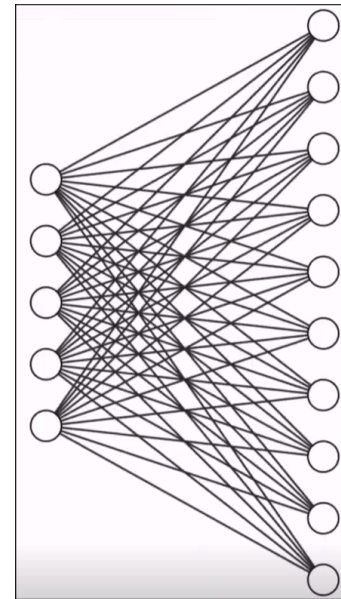


Figura 5. Ejemplo de múltiples entradas

|           |           |           |     |             |             |           |
|-----------|-----------|-----------|-----|-------------|-------------|-----------|
| $w_{0,0}$ | $w_{0,1}$ | $w_{0,2}$ | ... | $w_{0,n-2}$ | $w_{0,n-1}$ | $w_{0,n}$ |
| $w_{1,0}$ | $w_{1,1}$ | $w_{1,2}$ | ... | $w_{1,n-2}$ | $w_{1,n-1}$ | $w_{1,n}$ |
| ...       | ...       | ...       | ... | ...         | ...         | ...       |
| $w_{j,0}$ | $w_{j,1}$ | $w_{j,2}$ | ... | $w_{j,n-2}$ | $w_{j,n-1}$ | $w_{j,n}$ |

Figura 6. Matriz de pesos de una capa

regresión logística, o una multinomial que permita clasificar todas las clases que se estén evaluando.

A esto es a lo que se conoce como redes neuronales. Entre más capas intermedias se coloquen, más compleja será y más no linealidad evaluará. La cantidad de capas y de neuronas por capa es un hiperparámetro. Lo importante es que las capas sean derivables (por ejemplo, una sigmoide); si no, no se pueden utilizar, porque no se puede usar descenso del gradiente.

Finalmente se vio la solución para MNIST fully connected, con 2 capas intermedias y una final multinomial que clasifica los 10 números, como se muestra en la Figura 7.

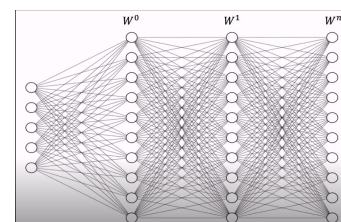


Figura 7. Solución Fully Connected de MNIST

## II. LECTURA DEL PROYECTO

### II-A. *Speech Commands*

Es una lectura que se asignó para el proyecto y describe el dataset con el que se trabajará.

### II-B. *SpecAugment*

Es una lectura que se asignó para el proyecto y describe cómo aumentar la cantidad de datos cuando no se tienen suficientes.

### II-C. *training keywords*

El paper es un buen ejemplo de lo que se puede seguir para el proyecto a nivel de estructura (por ejemplo, su sección de related work).

### II-D. *Objetivo*

Se comentó sobre como ONNX es una plataforma en la cual se puede migrar código para tenerlo en múltiples plataformas. Por lo tanto, en este proyecto se creará un modelo propio para reconocer comandos de voz y, a partir de ahí, migrarlo a un ambiente mobile con ONNX.

El profe aclara que no importa qué tipo de SO se utilice para la aplicación; puede ser iOS o Android, la versión que mejor se adapte al celular. Lo importante es que permita que el modelo sea integrado a una aplicación móvil.

El prototipo web HTML que dio el profe es una base muy sólida, pero puede cambiarse de acuerdo con las preferencias que se tengan, siempre y cuando se cumpla con los requerimientos.

### II-E. *Command Selection*

Se van a utilizar 10 comandos. En el dataset hay 35, entonces se tiene que hacer la selección de los indicados en la tarea, los cuales son: yes, no, up, down, left, right, on, off, stop, go.

### II-F. *Concepto App Móvil*

Se puede variar la aplicación como se guste; lo importante es que sí funcione la parte de integración con inteligencia artificial.

Es importante aclarar que no se van a usar redes neuronales densas; se usará otro tipo de modelos llamados Redes Neuronales Convolucionales (CNNs). Corresponde investigar sobre dichos modelos por cuenta propia, debido a que aún no se han repasado en clase.

La app no debe depender de servicios externos. Lo que se tiene que hacer es, básicamente, guardar los pesos obtenidos en el entrenamiento en PyTorch y pasarlos a ONNX para de ese modo tener el modelo de forma mobile.

### II-G. *Descripción del dataset*

El dataset consiste de múltiples audios, por lo que se comenta que el audio por sí solo tiene la naturaleza de ser temporal. Por lo que esa temporalidad se va a convertir en imágenes llamadas espectrogramas. Estos resumen las frecuencias en los audios y, gracias a ello, se podrán detectar patrones en los audios por medio de CNNs.

### II-H. *Diseño de arquitectura de IA*

Los modelos deben ser escritos a mano; solo se podrá usar `torch.nn` (la biblioteca base para redes neuronales de PyTorch).

En el notebook deben venir todas las pruebas, y se deben usar los mismos datasets para así garantizar reproducibilidad. Es básicamente lo que se ha hecho en tareas anteriores.

El modelo A es el más clásico de CNNs (LeNet-5); es muy corto, tiene pocas capas y suele dar buenos resultados. En el paper original las entradas eran de 32, y el profe dice que para este deben ser de 128 al menos.

En el modelo B se debe usar una arquitectura distinta, pero se debe justificar cuál se va a elegir. En la sección de metodología debe ir una explicación de los modelos que se evaluaron, y a partir de ahí indicar por qué se seleccionó dicho modelo como el más óptimo.

Puede que se quiera proponer otro modelo creado para el B. Sin embargo, no puede ser solo un LeNet-5 aumentado, realmente debe demostrarse por qué es diferente y funcional.

### II-I. *Preprocesamiento*

Se va a trabajar con dos conjuntos de datos: uno de datos crudos, es decir, tomar Speech Commands, pasar todo a espectrograma y entrenar con eso. Y el segundo será el mismo proceso, pero se debe aumentar la data usando el paper de SpecAugment. Por ejemplo, se va a pasar de tener 50 samples por clase a tener 300 samples. No obstante, hay que tener cuidado, porque si se pone una cantidad excesiva de augmentation, entonces los modelos durarán mucho entrenándose; pero, si no se hace, entonces se incumplirá con el requerimiento de hacer data augmentation.

Por ello se van a entrenar 4 modelos: modelo A crudo, modelo A aumentado, modelo B crudo y modelo B aumentado. Son 3 entrenamientos por cada modelo, para un total de 12 entrenamientos. En Google Colab seguramente se dure alrededor de 20 minutos por entrenamiento, así que se debe tomar el tiempo necesario..

No se debe hacer con CPU; en ese caso dura horas, por lo que es mejor hacerlo en Colab con GPU.

### II-J. *Evaluación de resultados*

Hay que llevar registro de bastantes estadística, pero lo más importante es que se lleve registro del comportamiento de la Loss Function y la de Validación. Esto se debe ir llevando con la plataforma Weights and Biases <https://wandb.ai/site/>. Esta plataforma se encarga de ir llevando recuento del entrenamiento y generando los gráficos. Por lo tanto, en esta etapa no hace falta usar Matplotlib para los gráficos. Las métricas como accuracy, recall, precision y f1-score son muy importantes.

### II-K. *Exportación del modelo*

Se debe elegir el modelo que mejores resultados obtuvo y exportarlo a ONNX.

## *II-L. Datos adicionales sobre el proyecto*

Es importante una sección de related work como la del paper de training keywords, toda la literatura con respecto a lo que se está viendo.

A día de hoy puede que CNNs ya no sea lo mejor; ya que existen otras opciones como visual transformers. Por lo que se puede hacer alusión a ello en la sección de related work.